How to Use This Document

This document contains exact Cursor IDE prompts - copy each green prompt block, paste it into Cursor AI Chat (Cmd+L or Ctrl+L), and Cursor will generate the files. Follow the steps in order. Each step builds on the previous one. After completing all steps, running 'make dev' will start all 10 services and print a startup banner showing DB connection, Redis connection, and service URL for each.

> PREREQUISITES: Install Node 22 LTS, pnpm (npm i -g pnpm), Docker Desktop, Git. Open the rsl-cards folder in Cursor IDE. Create the root folder first: mkdir rsl-cards && cd rsl-cards && git init

Services Being Built

#	Service	Dev Port	QA Port	Prod Port	Primary Responsibility
1	auth-service	3001	4001	5001	JWT tokens, OAuth Google/Apple, 2FA, refresh tokens
2	user-service	3002	4002	5002	Profiles, dealer/consumer roles, platform connections
3	inventory-service	3003	4003	5003	Card inventory, cost basis, aging alerts, consignment
4	transaction-service	3004	4004	5004	BUY/SELL/TRADE flow, profit calculation, tax data
5	listing-service	3005	4005	5005	Multi-channel listing, eBay/Whatnot sync, sold webhooks
6	card-db-service	3006	4006	5006	Card search, Ximilar scan, eBay comps, price history
7	ai-narrative-service	3007	4007	5007	Event detection, Claude API, narrative generation
8	notification-service	3008	4008	5008	FCM push, email (Resend), price alerts, SMS
9	analytics-service	3009	4009	5009	P&L; reports, tax exports, dashboard data
10	admin-service	3010	4010	5010	User management, narrative review, system monitoring

STEP 1

Monorepo Bootstrap - Turborepo + pnpm Workspace

Run this ONCE before creating any service

Everything lives in one monorepo. Turborepo manages build pipelines and caching. pnpm workspaces link shared packages into every service automatically. Do this step first - all other steps depend on this structure existing.

Prompt 1 - Create Monorepo Root Structure

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create a production-grade Turborepo monorepo called rsl-cards using pnpm workspaces.

Create this exact folder structure:
rsl-cards/
  apps/                        (frontend - leave empty for now)
  services/                    (all 10 microservices)
  packages/
    shared-types/              (TypeScript interfaces shared across all services)
    shared-db/                 (Drizzle ORM schemas + migrations)
    shared-utils/              (profit calc, date helpers, currency formatters)
    shared-constants/          (platform fees, enums, service ports)
    shared-config/             (eslint, tsconfig base, zod env validator)
  infra/
    docker/                    (docker-compose files for dev/qa/prod)
    nginx/                     (API gateway config for dev/qa/prod)
  scripts/                     (utility scripts)
  .github/workflows/           (CI/CD pipelines)

Root files to create:
- pnpm-workspace.yaml: packages: ['apps/*', 'services/*', 'packages/*']
- turbo.json: pipelines for dev, build, test, lint, type-check
- package.json: root scripts using turbo, pnpm version pinned to 9.x
- .nvmrc: 22
- .gitignore: node_modules, dist, .env*, *.local, docker volumes, coverage
- Makefile: dev, qa, prod, down, logs, migrate, seed, test commands
- README.md: step-by-step setup instructions for new developers
```

Prompt 2 - Shared Packages

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Set up all shared packages in the packages/ directory.

packages/shared-types/src/index.ts - TypeScript interfaces:
  User: id, email, role (dealer|consumer|admin), createdAt, updatedAt
  Card: id, playerName, year, setName, cardNumber, variation, grade, sport
  InventoryItem: id, userId, cardId, playerName, year, setName, grade,
    costBasis, currentMarketValue, quantity, isConsignment, listingStatus,
    daysHeld, addedAt, photos (string[]), sport, notes
  Transaction: id, userId, inventoryId, type (buy|sell|trade),
    channel, price, costBasis, profit, platformFee, paymentMethod,
    dealRating (good_deal|fair_price|overpaying), createdAt
  Listing: id, inventoryId, userId, platform, platformListingId,
    status (active|sold|ended|pending), listPrice, platformFeePct, netToDealer
  Notification: id, userId, type, title, body, read, createdAt
  ApiResponse<T>: success (boolean), data (T), error (string|null), message (string)
```

```
  HealthCheck: status (ok|error), service, environment, uptime, timestamp,
    checks: { database: {status, latency_ms}, redis: {status, latency_ms} }

packages/shared-constants/src/index.ts:
  PLATFORM_FEES: { ebay: 0.1285, whatnot: 0.08, mercari: 0.1, tcgplayer: 0.1025, shopify: 0.02 }
  SERVICE_PORTS: { dev: {auth:3001,...}, qa: {auth:4001,...}, prod: {auth:5001,...} }
  DEAL_RATING_THRESHOLDS: { good_deal: 0.85, fair_price: 1.05 }
  JWT_EXPIRY: { access: '15m', refresh: '7d' }
  CACHE_TTL: { comps: 900, search: 300, dashboard: 300, narratives: 21600 }

packages/shared-config/src/env.ts:
  Use zod to define and validate ALL required environment variables.
  Export: validateEnv() function that throws with clear message if any var is missing.
  Export: getEnv() function that returns typed env object.

packages/shared-utils/src/index.ts:
  calculateProfit(sellPrice, costBasis, platformFeePercent): number
  calculateDealRating(buyPrice, compPrice): DealRating
  formatCurrency(amount: number): string
  getDaysHeld(addedAt: Date): number
```

Prompt 3 - Shared DB (Drizzle Schema for All Tables)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In packages/shared-db/, install drizzle-orm, drizzle-kit, pg, @types/pg.

Create complete Drizzle PostgreSQL schemas in packages/shared-db/src/schema/:

users.ts: id (uuid pk), email (varchar 255 unique), password_hash (varchar),
  role (enum: dealer|consumer|admin default consumer), oauth_provider (varchar),
  oauth_id (varchar), refresh_token_hash (varchar), two_factor_secret (varchar),
  two_factor_enabled (boolean default false), fcm_token (varchar),
  created_at (timestamp defaultNow), updated_at (timestamp)

profiles.ts: id (uuid pk), user_id (uuid FK users), display_name (varchar 255),
  avatar_url (text), bio (text), location (varchar 255), sport_preference (varchar),
  business_name (varchar), tax_id (varchar), payment_methods (jsonb),
  connected_platforms (jsonb), created_at, updated_at

inventory.ts: (full schema - all fields from the architecture document)
  Include: computed days_held using sql template literal
  Include: listed_platforms as text array

transactions.ts: (full schema)
  channel as pgEnum with all values
  payment_method as pgEnum

customers.ts: id, user_id, name, email, phone, notes, total_purchased,
  total_spent, first_seen_at, created_at

listings.ts: (full schema)

cards.ts: id, player_name, year, set_name, card_number, variation, sport,
  manufacturer, is_rookie, notes, created_at

price_history.ts: id, card_id (FK cards), avg_price, last_sale,
  high_90d, low_90d, source, recorded_at
```

```
narratives.ts: (full schema with narrative_type enum)

notifications.ts + price_alerts.ts

analytics_snapshots.ts: id, user_id, date, revenue, cogs, gross_profit,
  cards_bought, cards_sold, by_channel (jsonb), by_sport (jsonb), created_at

admin_logs.ts: id, admin_user_id, action, target_type, target_id,
  details (jsonb), ip_address, created_at

Create packages/shared-db/src/index.ts exporting all schemas.
Create three drizzle config files for dev/qa/prod environments.
Add migrate and seed scripts to package.json.

Create packages/shared-db/src/seed.ts:
  2 dealer accounts, 1 consumer, 1 admin (password: Test1234!)
  10 inventory items per dealer, 20 transactions, 5 listings
  Log 'Seeding complete - X records created' when done
```

STEP 2

Environment Files - dev / qa / production

Three environments, zero secrets in code, ever

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create the environment configuration system for 3 environments.

Create .env.example at monorepo root (this file IS committed to git - no real values):
  NODE_ENV=development
  LOG_LEVEL=debug

  DATABASE_URL=postgresql://rsl_user:password@localhost:5432/rslcards_dev
  DATABASE_URL_READ_REPLICA=postgresql://rsl_user:password@localhost:5433/rslcards_dev
  DB_POOL_MIN=2
  DB_POOL_MAX=10

  REDIS_URL=redis://localhost:6379
  REDIS_PASSWORD=

  JWT_PRIVATE_KEY=REPLACE_WITH_RS256_PRIVATE_KEY
  JWT_PUBLIC_KEY=REPLACE_WITH_RS256_PUBLIC_KEY
  JWT_ACCESS_EXPIRY=15m
  JWT_REFRESH_EXPIRY=7d

  INTERNAL_SERVICE_KEY=REPLACE_WITH_RANDOM_64_CHAR_STRING

  AUTH_SERVICE_PORT=3001
  USER_SERVICE_PORT=3002
  INVENTORY_SERVICE_PORT=3003
  TRANSACTION_SERVICE_PORT=3004
  LISTING_SERVICE_PORT=3005
  CARD_DB_SERVICE_PORT=3006
  AI_NARRATIVE_SERVICE_PORT=3007
  NOTIFICATION_SERVICE_PORT=3008
  ANALYTICS_SERVICE_PORT=3009
  ADMIN_SERVICE_PORT=3010

  XIMILAR_API_KEY=
  EBAY_CLIENT_ID=
  EBAY_CLIENT_SECRET=
  WHATNOT_API_KEY=
  ANTHROPIC_API_KEY=
  FIREBASE_SERVICE_ACCOUNT=
  RESEND_API_KEY=
  SENTRY_DSN=
  SPORTRADAR_API_KEY=

Create infra/docker/.env.dev, .env.qa, .env.prod with:
  dev:  localhost DB/Redis, LOG_LEVEL=debug, all ports 3001-3010
  qa:   internal Docker network DB/Redis, LOG_LEVEL=info, ports 4001-4010
  prod: AWS RDS + ElastiCache URLs (placeholder), LOG_LEVEL=warn, ports 5001-5010

Create scripts/generate-keys.sh that generates RS256 keypair using openssl
and prints them formatted for pasting into .env files.
```

```
Add to README: 'Run scripts/generate-keys.sh to generate JWT keys, then copy
.env.example to .env.development and fill in the values'
```

STEP 3

Docker Setup - All 3 Environments

One Dockerfile per service, three compose files

Prompt 5 - Dockerfile Template (Apply to ALL 10 Services)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create a production-grade multi-stage Dockerfile for services/auth-service/.
We will copy this pattern to all 10 services.

Stage 1 (deps): FROM node:22-alpine AS deps
  WORKDIR /app
  COPY package.json pnpm-lock.yaml ./
  RUN npm install -g pnpm && pnpm install --frozen-lockfile --prod

Stage 2 (builder): FROM node:22-alpine AS builder
  WORKDIR /app
  COPY package.json pnpm-lock.yaml ./
  RUN npm install -g pnpm && pnpm install --frozen-lockfile
  COPY . .
  RUN pnpm build

Stage 3 (runner): FROM node:22-alpine AS runner
  WORKDIR /app
  ENV NODE_ENV=production
  RUN addgroup -S appgroup && adduser -S appuser -G appgroup
  COPY --from=deps /app/node_modules ./node_modules
  COPY --from=builder /app/dist ./dist
  COPY --from=builder /app/package.json ./
  USER appuser
  EXPOSE 3000
  HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3     CMD wget -qO- http://l
ocalhost:3000/health || exit 1
  CMD ["node", "dist/server.js"]

Also create services/auth-service/.dockerignore:
  node_modules, .env*, src, *.test.ts, coverage, .git, README.md
```

Prompt 6 - Docker Compose Development

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create infra/docker/docker-compose.dev.yml for local development.

Services:
1. postgres-dev: postgres:16-alpine, port 5432:5432, DB=rslcards_dev,
   volume postgres_dev_data, healthcheck: pg_isready every 10s

2. postgres-read-dev: same image, port 5433:5432, DB=rslcards_dev
   (simulates read replica locally)

3. redis-dev: redis:7-alpine, port 6379:6379, --appendonly yes,
   volume redis_dev_data

4. For EACH of the 10 microservices:
   build: context: ../../services/{name}, dockerfile: Dockerfile
```

```
    container_name: rsl-{name}-dev
    ports: {dev_port}:3000
    env_file: .env.dev
    environment:
      - NODE_ENV=development
      - LOG_LEVEL=debug
    depends_on: postgres-dev: condition: service_healthy, redis-dev
    volumes:
      - ../../services/{name}/src:/app/src (hot reload)
      - ../../packages:/packages (shared packages)
    command: pnpm dev
    restart: unless-stopped
    networks: [rsl-dev]

5. nginx-dev: nginx:alpine, port 80:80,
    volume: ../../infra/nginx/dev.conf:/etc/nginx/nginx.conf:ro
    depends_on: all 10 services

networks: rsl-dev (driver: bridge)
volumes: postgres_dev_data, redis_dev_data
```

Prompt 7 - Docker Compose QA + Production

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create infra/docker/docker-compose.qa.yml for QA environment.

Differences from dev:
- No src volume mounts (runs built production images)
- Add pgbouncer: bitnami/pgbouncer on port 6432 (connection pooler)
  pool_mode=transaction, max_client_conn=100, default_pool_size=20
  All services connect to pgbouncer:6432, NOT directly to postgres
- NODE_ENV=qa, LOG_LEVEL=info in all services
- resource limits: each service: memory=512m, cpus=0.5
- restart: always
- postgres and redis NOT exposed externally (internal network only)
- env_file: .env.qa for all services
- network: rsl-qa (isolated from dev)

Create infra/docker/docker-compose.prod.yml for production:
- NO postgres or redis containers (uses AWS RDS + ElastiCache)
- NODE_ENV=production, LOG_LEVEL=warn
- resource limits: memory=1g, cpus=1.0
- deploy.replicas: 2 (always 2 instances per service)
- deploy.update_config: parallelism=1, delay=10s, order=start-first
- deploy.restart_policy: on-failure, delay=5s, max_attempts=3
- logging: driver=awslogs, options per-service log group
- Only nginx exposed on 80/443
- env_file: .env.prod
```

STEP 4

Base Fastify Service Template

The exact same pattern applied to all 10 services

Prompt 8 - Base Service Scaffold (auth-service example)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create a complete production-grade Fastify service scaffold for services/auth-service/.

Folder structure:
services/auth-service/
  src/
    app.ts                Fastify factory function (createApp)
    server.ts             Entry point - starts server, prints banner
    config/
      env.ts              Zod validation for this service's env vars
      db.ts               Drizzle + pg Pool with testConnection()
      redis.ts            ioredis with testConnection()
      queue.ts            BullMQ Queue + Worker base setup
    routes/
      health.routes.ts    GET /health
      index.ts            Registers all route groups
    plugins/
      cors.ts             @fastify/cors with env-based origins
      helmet.ts           @fastify/helmet security headers
      rate-limit.ts       @fastify/rate-limit with Redis store
      request-id.ts       Adds X-Request-Id to every request
    middleware/
      error-handler.ts    Global Fastify error handler (structured JSON errors)
      internal-auth.ts    Checks X-Service-Key for /internal/* routes
    types/
      index.ts            Local TypeScript types for this service
  test/
    health.test.ts        Vitest tests for health endpoint
  package.json
  tsconfig.json           Extends packages/shared-config/tsconfig.base.json
  Dockerfile              (from Step 3 template)
  .dockerignore

package.json:
  name: @rsl/auth-service
  dependencies: fastify@4, @fastify/cors, @fastify/helmet, @fastify/rate-limit,
    drizzle-orm, pg, ioredis, bullmq, zod, pino, pino-pretty, uuid, @rsl/shared-types,
    @rsl/shared-db, @rsl/shared-constants, @rsl/shared-config
  devDependencies: typescript, tsx, @types/node, @types/pg, vitest, drizzle-kit
  scripts:
    dev: tsx watch src/server.ts
    build: tsc --project tsconfig.json
    start: node dist/server.js
    test: vitest run
    test:watch: vitest
```

Prompt 9 - server.ts with Startup Banner

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/auth-service/src/server.ts, create the main entry point.

On startup it must do these steps in order:
1. Call validateEnv() from shared-config - throw with clear message if any var missing
2. Create Fastify instance with pino logger (pretty in dev, JSON in qa/prod)
3. Register all plugins (cors, helmet, rate-limit, request-id)
4. Register routes
5. Test PostgreSQL connection - if fails: log error and exit process
6. Test Redis connection - if fails: log error and exit process
7. Start Fastify server
8. On successful start, print this EXACT startup banner using console.log:

   ╔═══════════════════════════════════════════════════════╗
   ║  RSL Cards - auth-service                             ║
   ║  Environment : development                            ║
   ║  URL         : http://localhost:3001                  ║
   ║  Health      : http://localhost:3001/health           ║
   ║  PostgreSQL  : Connected (rslcards_dev)               ║
   ║  Redis       : Connected (localhost:6379)             ║
   ╚═══════════════════════════════════════════════════════╝

(adjust service name, port, and DB name dynamically from env)

Handle graceful shutdown on SIGTERM and SIGINT:
- Close Fastify server (stop accepting new requests)
- Drain in-flight requests (wait up to 10 seconds)
- Close DB pool
- Disconnect Redis
- Log: 'auth-service shut down gracefully'
- process.exit(0)

Use pino for all application logging (not console.log except for the banner).
In development: use pino-pretty with colorized output.
In qa/prod: plain JSON logs for CloudWatch ingestion.
```

Prompt 10 - Health Route (Real DB + Redis Ping)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/auth-service/src/routes/health.routes.ts, create:

GET /health
This endpoint ACTUALLY pings both database and Redis and measures latency.

Response (200 when healthy):
{
  "status": "ok",
  "service": "auth-service",
  "environment": "development",
  "version": "1.0.0",
  "uptime": 45.23,
  "timestamp": "2026-03-01T10:00:00.000Z",
  "checks": {
    "database": { "status": "ok", "latency_ms": 2, "database": "rslcards_dev" },
    "redis":    { "status": "ok", "latency_ms": 1, "version": "7.0.0" }
  }
}
```

```
Response (503 when unhealthy):
{
  "status": "error",
  "service": "auth-service",
  "checks": {
    "database": { "status": "error", "error": "Connection refused" },
    "redis":    { "status": "ok", "latency_ms": 1 }
  }
}

Database ping: measure time for SELECT 1 query.
Redis ping: measure time for PING command, also get SERVER INFO for version.
Set Fastify schema for this route so it appears in type definitions.
This route must respond in under 200ms to pass Docker health checks.
```

STEP 5

Service-Specific Prompts - All 10 Services

First real route + service-specific setup for each

PROCESS: For each service below: (1) Copy Prompts 8-10 with the service name changed. (2) Run the service-specific prompt below. (3) Run 'pnpm dev' in the service folder. (4) Check the startup banner appears with green checkmarks.

1 auth-service | Port 3001 / 4001 / 5001

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/auth-service/, after base scaffold is complete, add:

src/routes/auth.routes.ts - first proof-of-life route:

POST /auth/ping
Request: { "message": "hello" }
Response: {
  "service": "auth-service",
  "received": "hello",
  "environment": "development",
  "db_connected": true,
  "redis_connected": true,
  "timestamp": "2026-03-01T10:00:00.000Z"
}

db_connected = result of attempting SELECT 1 (true/false)
redis_connected = result of attempting PING (true/false)

Register route in src/routes/index.ts with prefix '/auth'
Final URL: POST http://localhost:3001/auth/ping

Also add src/utils/jwt.ts skeleton:
  signAccessToken(payload): string   - uses JWT_PRIVATE_KEY
  signRefreshToken(userId): string   - uses JWT_PRIVATE_KEY
  verifyToken(token): JwtPayload     - uses JWT_PUBLIC_KEY
  For now: if keys not set, use a temp HS256 with warning log
```

2 user-service | Port 3002 / 4002 / 5002

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/user-service/ (after base scaffold), add:

GET /users/ping
Response: {
  "service": "user-service",
  "environment": "development",
  "db_connected": true,
  "redis_connected": true,
  "message": "user-service is running",
  "timestamp": "..."
```

```
}

Also add src/middleware/internal-auth.ts:
  Prehandler that checks X-Service-Key header on all routes with /internal prefix.
  If header missing or wrong: return 401 { error: 'Unauthorized', message: 'Invalid service key' }
  If correct: allow request through
  Key compared against INTERNAL_SERVICE_KEY env var (constant time comparison to prevent timing atta
cks)

Register in routes/index.ts with prefix '/users'
Final URL: GET http://localhost:3002/users/ping
```

3 inventory-service | Port 3003 / 4003 / 5003

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/inventory-service/ (after base scaffold), add:

GET /inventory/ping
Response includes extra field: "bullmq_worker": "running"

Also set up BullMQ worker:
src/jobs/aging-alert.job.ts:
  - Create Queue('inventory-aging', { connection: redis })
  - Create Worker('inventory-aging', async (job) => {
      log: 'Processing aging alert job ' + job.id
      return { processed: true }
    })
  - On worker start: log 'Inventory aging worker started and listening'
  - On worker error: log error details

src/config/queue.ts: exports agingQueue and agingWorker

In server.ts: start the worker AFTER redis connects, include in startup banner:
  ■  BullMQ Worker : inventory-aging - Running    ■

Register routes with prefix '/inventory'
Final URL: GET http://localhost:3003/inventory/ping
```

4 transaction-service | Port 3004 / 4004 / 5004

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/transaction-service/ (after base scaffold), add:

GET /transactions/ping
Response includes: "queue_connected": true

Also create src/utils/profit.ts with pure calculation functions:
  calculateProfit(sellPrice: number, costBasis: number, platformFeePercent: number): number
    formula: sellPrice - costBasis - (sellPrice * platformFeePercent)

  calculateDealRating(buyPrice: number, compPrice: number): 'good_deal'|'fair_price'|'overpaying'
    good_deal: buyPrice <= compPrice * 0.85
    fair_price: buyPrice <= compPrice * 1.05
    overpaying: buyPrice > compPrice * 1.05
```

```
  calculatePlatformFee(price: number, platform: string): number
    uses PLATFORM_FEES from @rsl/shared-constants
    returns fee amount in dollars

  calculateMarginPercent(sellPrice: number, costBasis: number): number
    formula: ((sellPrice - costBasis) / costBasis) * 100

Write tests in test/profit.test.ts using vitest - test all edge cases.
These functions are financial calculations - they need 100% test coverage.

Also add queue config for the deactivation queue:
src/config/queue.ts: exports transactionQueue
```

5 listing-service | Port 3005 / 4005 / 5005

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/listing-service/ (after base scaffold), add:

GET /listings/ping
Response: standard ping response + "deactivation_worker": "running"

Add webhook receiver skeleton:
POST /listings/webhook/:platform
  - Extract platform from params (ebay|whatnot|mercari|tcgplayer)
  - Log: 'Webhook received from {platform}: ' + JSON.stringify(body)
  - Verify webhook signature header (platform-specific - for now just log it)
  - Return 200 { "received": true, "platform": platform }

Add cross-platform deactivation worker:
src/jobs/deactivation.job.ts:
  Queue: 'deactivate-listings' with priority support
  Worker: logs 'Deactivation job received for inventory: ' + job.data.inventoryId
  Priority 1 (highest): process immediately
  On start: log 'Cross-platform deactivation worker started'

Register routes: /listings prefix
Webhook URL: POST http://localhost:3005/listings/webhook/ebay
```

6 card-db-service | Port 3006 / 4006 / 5006

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/card-db-service/ (after base scaffold), add:

GET /cards/ping
Response: standard ping + "cache_enabled": true, "ximilar_configured": boolean

Add Redis cache utility src/utils/cache.ts:
  setCache(key: string, value: unknown, ttlSeconds: number): Promise<void>
  getCache<T>(key: string): Promise<T | null>
  invalidateCache(pattern: string): Promise<number>  (returns deleted key count)

  Cache key builders (export as constants):
    cardScanKey = (hash: string) => 'card:scan:' + hash
```

```
    cardCompsKey = (cardId: string) => 'card:comps:' + cardId
    cardSearchKey = (query: string) => 'card:search:' + encodeURIComponent(query)

Add specific rate limit for this service in plugins/rate-limit.ts:
  /cards/scan route: 60 requests per minute per user (key: userId from JWT)
  /cards/*/comps route: 30 requests per minute per user
  Return 429 with: { "error": "Rate limit exceeded", "retryAfter": seconds }

Add Ximilar client skeleton src/clients/ximilar.ts:
  identifyCard(imageBase64: string): Promise<CardIdentification>
  If XIMILAR_API_KEY not set: return mock card data + log 'Ximilar mock mode'

Register routes: /cards prefix
```

7 ai-narrative-service | Port 3007 / 4007 / 5007

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/ai-narrative-service/ (after base scaffold), add:

GET /narratives/ping
Response: standard ping + "cron_running": true, "claude_configured": boolean

Add BullMQ scheduler (cron) in src/jobs/ingestion.cron.ts:
  Create BullMQ QueueScheduler and Queue('narrative-ingestion')
  Add repeating job: { repeat: { cron: '0 */2 * * *' } }
  Worker logs: 'Narrative ingestion cron fired at: ' + new Date().toISOString()

DEV ONLY - add manual trigger route:
  GET /narratives/trigger-ingestion
  Manually adds one ingestion job to queue
  Response: { "triggered": true, "jobId": "..." }
  This allows testing without waiting 2 hours

Add Claude API client src/clients/claude.ts:
  generateNarrative(context: NarrativeContext): Promise<NarrativeOutput>
  If ANTHROPIC_API_KEY is set: call real API
  If not set: return mock narrative + log 'Claude API: mock mode (no API key)'
  Mock returns: { headline: 'Mike Trout cards up 15%', body: 'Mock narrative...' }

In startup banner add:
  ■  Cron Job     : narrative-ingestion - Every 2hrs ■
  ■  Claude API   : Mock mode (set ANTHROPIC_API_KEY) ■
```

8 notification-service | Port 3008 / 4008 / 5008

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/notification-service/ (after base scaffold), add:

GET /notifications/ping
Response: standard ping + "fcm_configured": boolean, "email_configured": boolean

fcm_configured = !!process.env.FIREBASE_SERVICE_ACCOUNT
email_configured = !!process.env.RESEND_API_KEY
```

```
Add priority notification worker src/jobs/notification.worker.ts:
  Queue: 'notifications' with 3 concurrency levels
  Worker processes jobs, checks job.opts.priority:
    priority 1 (sale, sold_alert): log 'CRITICAL notification: ' + type
    priority 2 (price_alert, narrative): log 'STANDARD notification: ' + type
    priority 3 (digest, reminder): log 'DIGEST notification: ' + type
  For now: just log the notification, don't send yet

Add FCM client skeleton src/clients/fcm.ts:
  sendPush(token: string, title: string, body: string, data?: object): Promise<void>
  If FIREBASE_SERVICE_ACCOUNT not set: log 'FCM mock: would send to ' + token.slice(0,10)
  If configured: will use firebase-admin SDK (install firebase-admin)

Add email client skeleton src/clients/email.ts:
  sendEmail(to: string, subject: string, html: string): Promise<void>
  If RESEND_API_KEY not set: log 'Email mock: would send to ' + to
  If configured: use Resend SDK (install resend)

In startup banner:
  ■  FCM          : Mock mode (no FIREBASE_SERVICE_ACCOUNT) ■
  ■  Email        : Mock mode (no RESEND_API_KEY)           ■
```

9 analytics-service | Port 3009 / 4009 / 5009

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/analytics-service/ (after base scaffold), add:

GET /analytics/ping
Response: standard ping PLUS:
  "primary_db_connected": true,
  "read_replica_connected": true

CRITICAL: This service needs TWO database connections:
  src/config/db.ts -> connects to DATABASE_URL (primary - only if writes ever needed)
  src/config/db-read.ts -> connects to DATABASE_URL_READ_REPLICA

All analytics query functions MUST import from db-read, never db.
In server.ts: test BOTH connections on startup. If either fails, log error and exit.

In startup banner show both:
  ■  PostgreSQL Primary : Connected              ■
  ■  PostgreSQL Replica : Connected              ■

Add nightly snapshot cron src/jobs/daily-snapshot.cron.ts:
  BullMQ: queue 'analytics-snapshots', runs at 2am daily (cron: '0 2 * * *')
  Worker: log 'Daily snapshot job started for: ' + date

DEV ONLY - manual trigger:
  GET /analytics/trigger-snapshot
  Fires one snapshot job immediately for testing

In startup banner:
  ■  Snapshot Cron: analytics-snapshots - Daily 2am ■
```

10 admin-service | Port 3010 / 4010 / 5010

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
In services/admin-service/ (after base scaffold), add:

GET /admin/ping
Response: standard ping response (admin auth bypassed for ping)

Add admin JWT middleware src/middleware/admin-auth.ts:
  Verify JWT from Authorization: Bearer header using JWT_PUBLIC_KEY
  Decode token, check role field === 'admin'
  If not admin: 403 { "error": "Forbidden", "message": "Admin role required" }
  If no token: 401 { "error": "Unauthorized", "message": "Token required" }
  Apply this middleware to ALL routes EXCEPT /health and /admin/ping

Add GET /admin/health-all route:
  Calls GET /health on ALL other 9 services in parallel (Promise.allSettled)
  Service URLs built from SERVICE_PORTS[NODE_ENV] in shared-constants
  Response: {
    "overall": "ok",   (ok if all healthy, degraded if some failing, down if all failing)
    "services": {
      "auth-service":        { "status": "ok", "latency_ms": 5 },
      "user-service":        { "status": "ok", "latency_ms": 4 },
      "inventory-service":   { "status": "error", "error": "Connection refused" },
      ...
    },
    "healthy": 9,
    "unhealthy": 1,
    "timestamp": "..."
  }
  Returns 200 even if some services are down (the body shows what is failing)
```

STEP 6

Nginx API Gateway + Migration Commands

Route all traffic, run database migrations

Prompt 21 - Nginx Config (All 3 Environments)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create infra/nginx/dev.conf - Nginx for local development.

Listen on port 80. Route by URL prefix:
  /auth/*          -> http://auth-service:3001
  /users/*         -> http://user-service:3002
  /inventory/*     -> http://inventory-service:3003
  /transactions/*  -> http://transaction-service:3004
  /listings/*      -> http://listing-service:3005
  /cards/*         -> http://card-db-service:3006
  /narratives/*    -> http://ai-narrative-service:3007
  /notifications/* -> http://notification-service:3008
  /analytics/*     -> http://analytics-service:3009
  /admin/*         -> http://admin-service:3010

Headers to add on all proxy requests:
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  proxy_set_header X-Request-ID $request_id;

Rate limiting zones:
  general: $binary_remote_addr zone=api:10m rate=100r/m burst=20
  auth: $binary_remote_addr zone=auth_limit:10m rate=10r/m burst=5
  Apply auth zone to /auth/login and /auth/register locations

Health check route: GET /nginx-health returns 200 OK text

Create infra/nginx/qa.conf (same routing, add access logs)
Create infra/nginx/prod.conf (same + larger buffers, gzip compression,
  add security headers: HSTS, X-Content-Type-Options, X-Frame-Options)
```

Prompt 22 - Migration + Verification Commands

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Add migration and verification tooling.

In packages/shared-db/package.json add scripts:
  generate:     drizzle-kit generate
  migrate:dev:  drizzle-kit migrate --config drizzle.dev.config.ts
  migrate:qa:   drizzle-kit migrate --config drizzle.qa.config.ts
  migrate:prod: drizzle-kit migrate --config drizzle.prod.config.ts
  seed:dev:     tsx src/seed.ts (guard: throw if NODE_ENV !== development)
  studio:       drizzle-kit studio

In root package.json add:
  db:migrate: pnpm --filter @rsl/shared-db migrate:dev
  db:seed:    pnpm --filter @rsl/shared-db seed:dev
  db:studio:  pnpm --filter @rsl/shared-db studio
```

```
Create scripts/verify-all-services.sh:
  After 'make dev', this script checks each service is up:
  1. Wait up to 60 seconds for all services to be healthy
  2. Hit each service's /health endpoint
  3. Print pass/fail table for all 10 services
  4. Print overall: 'X/10 services healthy'
  5. Exit 0 if all healthy, exit 1 if any failing
```

STEP 7

GitHub Actions CI/CD Pipelines

Automated testing + deployment to all 3 environments

Prompt 23 - CI Pipeline (Pull Request Checks)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create .github/workflows/ci.yml - runs on every pull request.

Trigger: pull_request targeting main or develop branch

Job 1 - lint-and-typecheck:
  runs-on: ubuntu-latest
  steps: checkout, setup-node 22, pnpm install with cache,
  run: pnpm turbo lint && pnpm turbo type-check

Job 2 - test:
  runs-on: ubuntu-latest
  services:
    postgres: image: postgres:16, env: POSTGRES_DB=rslcards_test
    redis: image: redis:7
  steps: checkout, setup-node 22, pnpm install,
  run: pnpm turbo test
  Upload coverage to Codecov

Job 3 - build:
  runs-on: ubuntu-latest
  steps: checkout, setup-node 22, pnpm install,
  run: pnpm turbo build
  Verify all 10 service dist/ folders created

Job 4 - docker-build:
  strategy.matrix.service: [auth, user, inventory, transaction, listing, card-db, ai-narrative, noti
fication, analytics, admin]
  steps: checkout, Docker Buildx setup,
  run: docker build -t rsl-${{ matrix.service }}:test ./services/${{ matrix.service }}-service
  (just verify it builds, don't push)

All 4 jobs must pass for PR to merge (add branch protection rule note in README)
```

Prompt 24 - CD Pipeline (Deploy to QA + Production)

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create .github/workflows/deploy.yml for automated deployment.

On push to develop branch -> deploy to QA:
1. Run reusable CI workflow (all checks must pass)
2. Configure AWS credentials from GitHub Secrets
3. Build and push Docker images to AWS ECR
   Tag: {service}:qa-{short_sha}
   Push all 10 service images in parallel
4. Run DB migrations against QA DB:
   ssh to bastion or use ECS exec to run drizzle-kit migrate
5. Update ECS Fargate services (blue/green):
   aws ecs update-service for each of 10 services
```

```
6. Wait for ECS deployment to stabilize (max 5 min)
7. Smoke test: curl each service /health via QA load balancer
8. Slack notification: success or failure with commit info

On git tag push v*.*.* -> deploy to Production:
1. Same as QA pipeline
2. Add manual approval gate: GitHub Environment 'production' with required reviewers
3. Deploy services one at a time (not all in parallel - safer)
4. Rollback on failure: if health check fails after 3 min, redeploy previous image
5. Slack notification with release tag and changelog

Required GitHub Secrets to document in README:
  AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION,
  ECR_REGISTRY, ECS_CLUSTER_QA, ECS_CLUSTER_PROD, SLACK_WEBHOOK_URL
```

STEP 8

Turborepo Pipeline + Final Architecture Rules

One command to run everything - rules that never get broken

Prompt 25 - turbo.json Full Configuration

CURSOR PROMPT (paste into Cursor AI Chat - Cmd+L)

```
Create the complete turbo.json at monorepo root.

{
  "$schema": "https://turbo.build/schema.json",
  "globalEnv": ["NODE_ENV", "DATABASE_URL", "REDIS_URL"],
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**"],
      "cache": true
    },
    "dev": {
      "dependsOn": ["^build"],
      "cache": false,
      "persistent": true
    },
    "test": {
      "dependsOn": ["^build"],
      "outputs": ["coverage/**"],
      "cache": true,
      "env": ["NODE_ENV"]
    },
    "lint": { "cache": true },
    "type-check": { "cache": true },
    "db:migrate": { "cache": false },
    "db:seed": { "cache": false }
  }
}

Update root package.json scripts:
  dev:              turbo dev
  build:            turbo build
  test:             turbo test
  test:coverage:    turbo test -- --coverage
  lint:             turbo lint
  type-check:       turbo type-check
  dev:auth:         turbo dev --filter=@rsl/auth-service
  dev:user:         turbo dev --filter=@rsl/user-service
  dev:inventory:    turbo dev --filter=@rsl/inventory-service
  dev:transaction:  turbo dev --filter=@rsl/transaction-service
  dev:listing:      turbo dev --filter=@rsl/listing-service
  dev:cards:        turbo dev --filter=@rsl/card-db-service
  dev:narrative:    turbo dev --filter=@rsl/ai-narrative-service
  dev:notification: turbo dev --filter=@rsl/notification-service
  dev:analytics:    turbo dev --filter=@rsl/analytics-service
  dev:admin:        turbo dev --filter=@rsl/admin-service
```

Architecture Rules - Never Break These

RULE 1 - Services never share databases directly. Each service only touches its own tables. Analytics Service reads from shared read replica but NEVER writes. No service should import another service's DB config.

RULE 2 - No secrets ever in code or git. All sensitive values in .env files (gitignored). .env.example is the only env file committed. Production uses AWS Secrets Manager. If you see a real API key in a commit - rotate it immediately.

RULE 3 - All /internal/* routes must verify X-Service-Key header. This prevents external callers from hitting service-to-service endpoints. Use constant-time string comparison to prevent timing attacks.

RULE 4 - Every service must have /health that actually pings DB and Redis. Docker, load balancers, and the admin service all depend on it. It must respond in under 200ms and return 503 if any dependency is down.

RULE 5 - Analytics Service uses only the read replica for all SELECT queries. DATABASE_URL_READ_REPLICA in db-read.ts. Never import from db.ts in analytics queries.

RULE 6 - Heavy jobs are always async via BullMQ. Tax exports, AI generation, notification dispatch - never block an HTTP response for these. Return job_id immediately.

RULE 7 - Three environments are always maintained. Every feature goes dev -> qa -> prod. Never skip QA. Never test in production. Never use production DB for development.

Final Folder Structure After All Prompts Complete

```
rsl-cards/
├── services/
│   ├── auth-service/          src/{app,server,config,routes,plugins,middleware}/ test/ Dockerfile
│   ├── user-service/          src/{app,server,config,routes,plugins,middleware}/ test/ Dockerfile
│   ├── inventory-service/     src/.../jobs/aging-alert.job.ts   Dockerfile
│   ├── transaction-service/   src/.../utils/profit.ts test/profit.test.ts  Dockerfile
│   ├── listing-service/       src/.../jobs/deactivation.job.ts  Dockerfile
│   ├── card-db-service/       src/.../utils/cache.ts clients/ximilar.ts  Dockerfile
│   ├── ai-narrative-service/  src/.../jobs/ingestion.cron.ts clients/claude.ts  Dockerfile
│   ├── notification-service/  src/.../jobs/notification.worker.ts clients/{fcm,email}.ts  Dockerfile
│   ├── analytics-service/     src/config/db-read.ts jobs/daily-snapshot.cron.ts  Dockerfile
│   └── admin-service/         src/middleware/admin-auth.ts routes/admin.routes.ts  Dockerfile
├── packages/
│   ├── shared-types/          src/index.ts  (all interfaces)
│   ├── shared-db/             src/schema/*.ts seed.ts  drizzle.*.config.ts
│   ├── shared-utils/          src/index.ts  (profit, date, currency)
│   ├── shared-constants/      src/index.ts  (fees, ports, enums, ttl)
│   └── shared-config/         src/env.ts  tsconfig.base.json  eslint.config.js
├── infra/
│   ├── docker/
│   │   ├── docker-compose.dev.yml
│   │   ├── docker-compose.qa.yml
│   │   ├── docker-compose.prod.yml
│   │   └── .env.dev  .env.qa  .env.prod
│   └── nginx/
│       └── dev.conf  qa.conf  prod.conf
├── scripts/
│   ├── verify-all-services.sh
│   └── generate-keys.sh
├── .github/workflows/
│   ├── ci.yml
│   └── deploy.yml
├── .env.example          <- committed to git (no real values)
├── .env.development      <- gitignored (your real dev values)
└── .gitignore
```

```
■■■ turbo.json
■■■ pnpm-workspace.yaml
■■■ package.json
■■■ Makefile
■■■ README.md
```

Verification - What to Check After Each Service Starts

Check	Command	Expected Result
Startup banner	make dev	All 10 services print banner with green DB/Redis status
Health endpoints	curl localhost:3001/health	{"status":"ok","checks":{"database":{"status":"ok"},...}}
Ping routes	curl -X POST localhost:3001/auth/ping	{"service":"auth-service","db_connected":true,...}
Nginx gateway	curl localhost:80/auth/auth/ping	Same response routed through nginx
Admin health-all	curl localhost:3010/admin/ping	Returns status for all 10 services
DB migrations	pnpm db:migrate	All tables created, 0 errors
Seed data	pnpm db:seed	X records created across all tables
DB studio	pnpm db:studio	Drizzle Studio opens at localhost:4983
All tests pass	pnpm turbo test	All vitest tests green including profit.test.ts
TypeScript clean	pnpm turbo type-check	0 TypeScript errors across all services